GITHUB FLOW

Flujo de trabajo colaborativo

Objetivos:

- Verificar que Git esté instalado y configurado correctamente.
- Comprender qué problema resuelve GitHub Flow y porqué importa.
- Conocer conceptos claves: repositorio, rama, commit, remote.

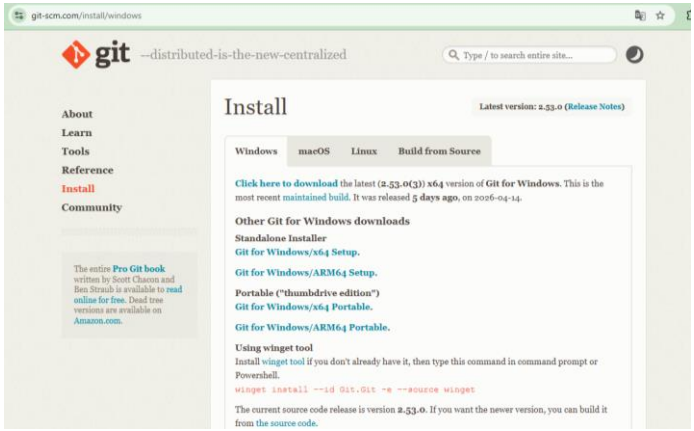
1. Instalación de Git

a. Verificaremos si tenemos instalado git:

   i. Desde un terminal (cmd, powershell, terminal), digitar: **git –version**

b. En caso que no se encuentre instalado, acceder a: **git-scm.com/download/win:**

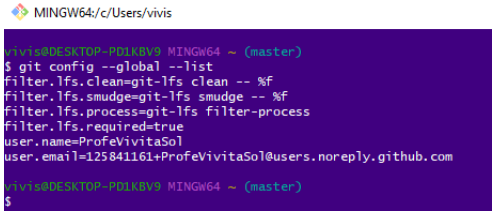
   i. Descargar y ejecutar el instalador.
   ii. Todas las opciones de la aplicación dejar por defecto.
   iii. Desde git bash, revisar la versión instalada



2. Configuración inicial

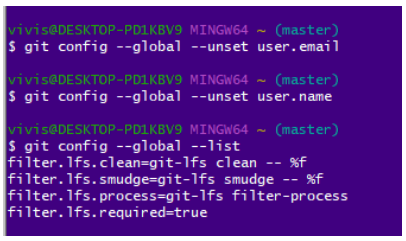
a. Git necesita conocer quién es el usuario que "firmará" los "commits". Desde consola, ejecutar los siguientes comandos:

   i. **git config --global user.name "Tu Nombre Apellido"**
   ii. **git config --global user.email tu@email.com**
   iii. Verificamos que ha quedado almacenado: git config –global –list



   iv. Si se necesita borrar una cuenta de la máquina:
   git config --global --unset user.email
   git config –global –unset user.name

   Posteriormente se configura la cuenta con un nuevo email o usuario.



3. Conceptos básicos

a. Repositorio: carpeta del proyecto + todo el historial de cambios
b. Commit: fotografía del estado del proyecto en un determinado momento.
c. Rama: línea de desarrollo independiente dentro del mismo repositorio.
d. Remote: copia del repositorio en un servidor (GitHub).
e. Main/master: rama principal – código estable del proyecto

4. GitHub Flow:

Es un flujo de trabajo basado en ramas que permite trabajar de forma ordenada y segura. La idea principal corresponde a:

a. La rama principal siempre tiene codificación que funciona y está listo para producción.
b. Todo nuevo trabajo se realiza en una rama separada.
c. La rama se integra main sólo después de revisión (Pull request).

Flujo de trabajo

a. **Clonar o actualizar el repositorio**: git clone / git pull
b. **Crear una nueva rama:** git switch -c nombre-rama: no se trabaja directamente en el main. Cada tarea es igual a una nueva rama.
c. **Realizar cambios y commits**: git add . / git commit: almacenar avances frecuentes con mensajes descriptivos.
d. **Crear pull request (en github.com)**: solicita que la rama se integre a main.
e. **Integrar a main y eliminar la rama:** git switch main / git pull

Nota:

- **Git switch nombre-rama**: permite cambiar de rama.
- **Git switch -c nueva-rama**: permite crear una rama y cambiarnos a la rema creada.

5. <u>PRACTICANDO</u>

5.1 Creamos un repositorio en github como público: practicaGitFlow
5.2 Desde nuestro equipo clonamos el repositorio creado:
git clone https://github.com/ProfeVivitaSol/practicaGithubFlow.git

```
C:\Users\vivis\OneDrive\Desktop>git clone https://github.com/ProfeVivitaSol/practicaGithubFlow.git
Cloning into 'practicaGithubFlow'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

C:\Users\vivis\OneDrive\Desktop>
```

5.3 Ingresamos a la carpeta de nuestro repositorio
5.4 Verifiquemos en que rama nos encontramos: **git branch**

```
C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git branch
* main

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>
```

5.5 Creando ramas: los nombres más utilizados de manera estándar y profesional

- features/ : nuevas funcionalidades
- fix/ : corrección de errores
- hotfix/ : correcciones importantes en producción
- docs/ : documentación
- refactor/ : mejoras internas sin cambiar el comportamiento
- test/ : pruebas unitarias, integración
- chore/ : tareas técnicas (configuración, dependencias, etc)
- perf/ : mejoras de rendimiento
- build/ : cambios en build/deploy

Ejemplos:

- feature/login-con-google
- fix/error-validacion-formulario
- hotfix/crash-al-iniciar-app
- docs/actualizar-readme
- refactor/separar-logica-autenticacion
- chore/actualizar-dependencias

Normas importantes:

- Todo se escribe en minúsculas
- Sin espacios, utilizar –
- Descripciones cortas, pero claras
- Una rama = un objetivo

Adicionalmente, se puede tener ramas principales, como: main, develop.

Crearemos la siguiente rama: feature/mi-primera-seccion
Verificamos que nos encontramos en la rama creada

```
C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git branch
* main

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git switch -c feature/mi-primera-seccion
Switched to a new branch 'feature/mi-primera-seccion'

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git branch
* feature/mi-primera-seccion
  main

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>
```

- Haciendo cambios en la rama: modificaremos el archivo readme.md.
- Digitamos notepad readme.md
- Modificamos el contenido del archivo, grabamos y cerramos notepad.

```
C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git branch
* feature/mi-primera-seccion
  main

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>notepad readme.md
```

5.6 Revisamos el estado de nuestro repositorio, es decir, ver que archivos cambiaron:

```
C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git status
On branch feature/mi-primera-seccion
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>
```

5.7 Agregamos los cambios al área de staging, digitamos: git add .
5.8 Verificamos el estado actual, digitando git status, observa cómo cambia el estado.
5.9 Creamos commit con mensaje descriptivo:

git commit -m "docs: agrego sección de introducción al README"

```
C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git status
On branch feature/mi-primera-seccion
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md


C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git commit -m "docs: agrego sección de introducción al README"
[feature/mi-primera-seccion 376baf6] docs: agrego sección de introducción al README
 1 file changed, 2 insertions(+)
```

Notas importantes sobre commit:

- Utilizar prefijo descriptivo: feat, fix, docs, refactor, test.
- Escribir en imperativo: “agrego”, “corrijo”, “actualizo”
- Ejemplo correcto: feat: agrego validación de email en formulario de registro”

5.10 Subir rama a github

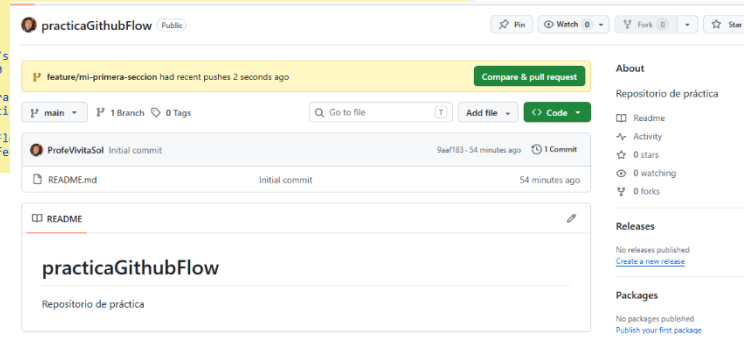
- **Publicamos rama en github:** git push origin feature/mi-primera-seccion

En el primer intento la consola genero error, porque git detecto que estaba vinculada la cuenta a otro usuario. ¿Qué se debe realizar? Eliminar las credenciales desde Windows asociadas a github (Administrador de credenciales).

```
C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git commit -m "docs: agrego sección de introducción al README"
[feature/mi-primera-seccion 376baf6] docs: agrego sección de introducción al README
 1 file changed, 2 insertions(+)

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git push origin feature/mi-primera-seccion
remote: Permission to ProfeVivitaSol/practicaGithubFlow.git denied to ViviSolProfe.
fatal: unable to access 'https://github.com/ProfeVivitaSol/practicaGithubFlow.git/': The requested URL returned error: 4
03

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git push origin feature/mi-primera-seccion
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 360 bytes | 360.00 KiB/s
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'feature/mi-primera
remote:      https://github.com/ProfeVivitaSol/practi
remote:
To https://github.com/ProfeVivitaSol/practicaGithubFl
 * [new branch]      feature/mi-primera-seccion -> fe
```



Nota importante:

Token de acceso personal

GitHub ya no acepta contraseñas por terminal. Debemos configurar token de acceso, configurándolo desde: GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic)

Generamos un token con permiso 'repo' y lo almacenamos en un lugar seguro.

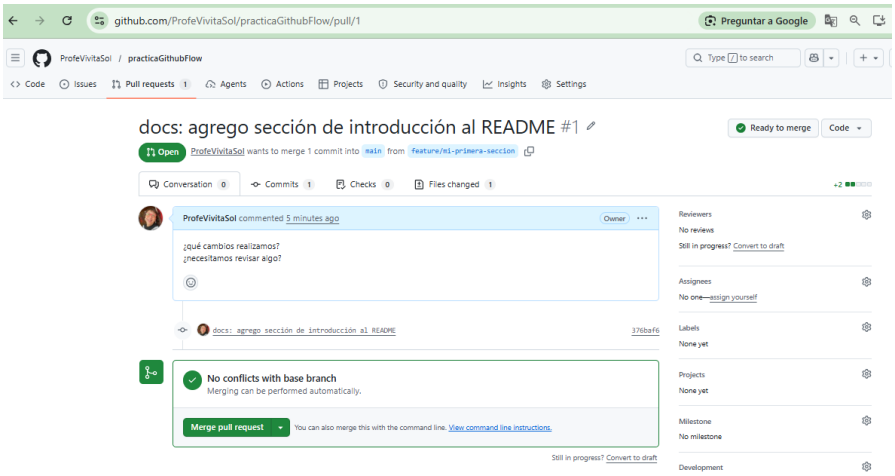
Cuando Git pida contraseña, ingresamos el token en vez de tu contraseña.

6. **Pull Request**: es una solicitud formal para integrar los cambios de una rama a “main”. Se conoce como el corazón de la colaboración en Github Flow:

- **Permite revisión:** otros pueden comentar, sugerir cambios o aprobar
- **Muestra el diff:** GitHub muestra exactamente qué líneas cambiaron
- **Activa CI/CD:** en proyectos reales, dispara pruebas automáticas
- **Genera historial:** queda registro de por qué se hizo cada cambio

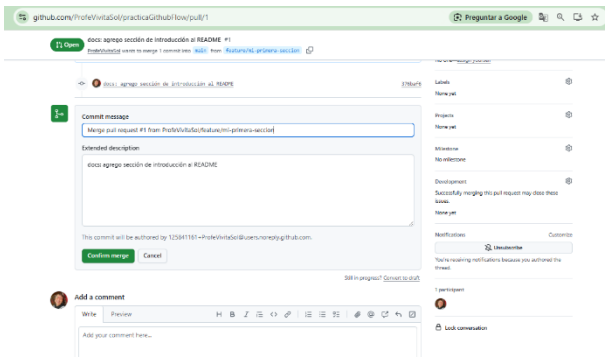
6.1 Creamos pull request:

- En github.com, en el repositorio creado,
- Seleccionar la opción “Compare & pull request”
- En el campo título ingresar: **'Agrego sección de introducción al README'**
- Descripción: **¿Qué cambio? ¿Es necesario revisar algo?**
- Base Branch: main
- Compare Branch: feature/mi-primera-sección
- Seleccionar la opción créate pull request

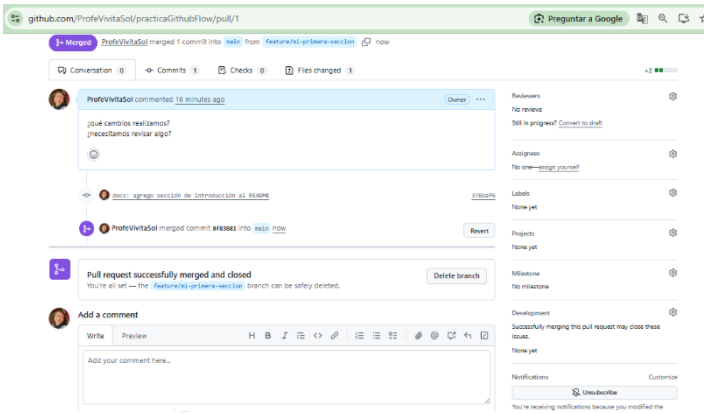


6.2 Revisar y realizar merge

- Cuando se trabaja en equipo, otro integrante revisar “Pull Request”.
- En nuestro caso, usted realizará el “merge”.
- Primero que todo, ingresamos a Pull Request y seleccionamos la opción Files changed.
- Revisamos el “diff” que sea correcto. El diff nos permite comparar entre dos versiones de un archivo. En esta pestaña, “Files changed” el diff nos indica: “líneas en verde (+)” nos indica lo que se agregó, “líneas en rojo (-)” lo que hemos eliminado, “sin color” lo que no ha cambiado.
- Una vez que hemos revisado el “diff”, regresamos a la pestaña “conversation” y hacemos clic en **"Merge pull request".**



- Posteriormente, confirmamos en “confirm merge”.



- Una vez realizado merge, podemos seleccionar “Delete Branch” para limpiar la rama.

7. Actualizar la copia local
   - Regresamos a main: git switch main.
   - Descargamos los cambios que se integraron a través de Pull Request: git pull origin main.

```
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 944 bytes | 157.00 KiB/s, done.
From https://github.com/ProfeVivitaSol/practicaGithubFlow
 * branch            main       -> FETCH_HEAD
   9aaf183..8f83881  main       -> origin/main
Updating 9aaf183..8f83881
Fast-forward
 README.md | 2 ++
 1 file changed, 2 insertions(+)
```

   - Eliminamos la rama local: git Branch -d feature /mi-primera-seccion
   - Verificamos que la rama main tenga el contenido nuevo: git log --oneline -5

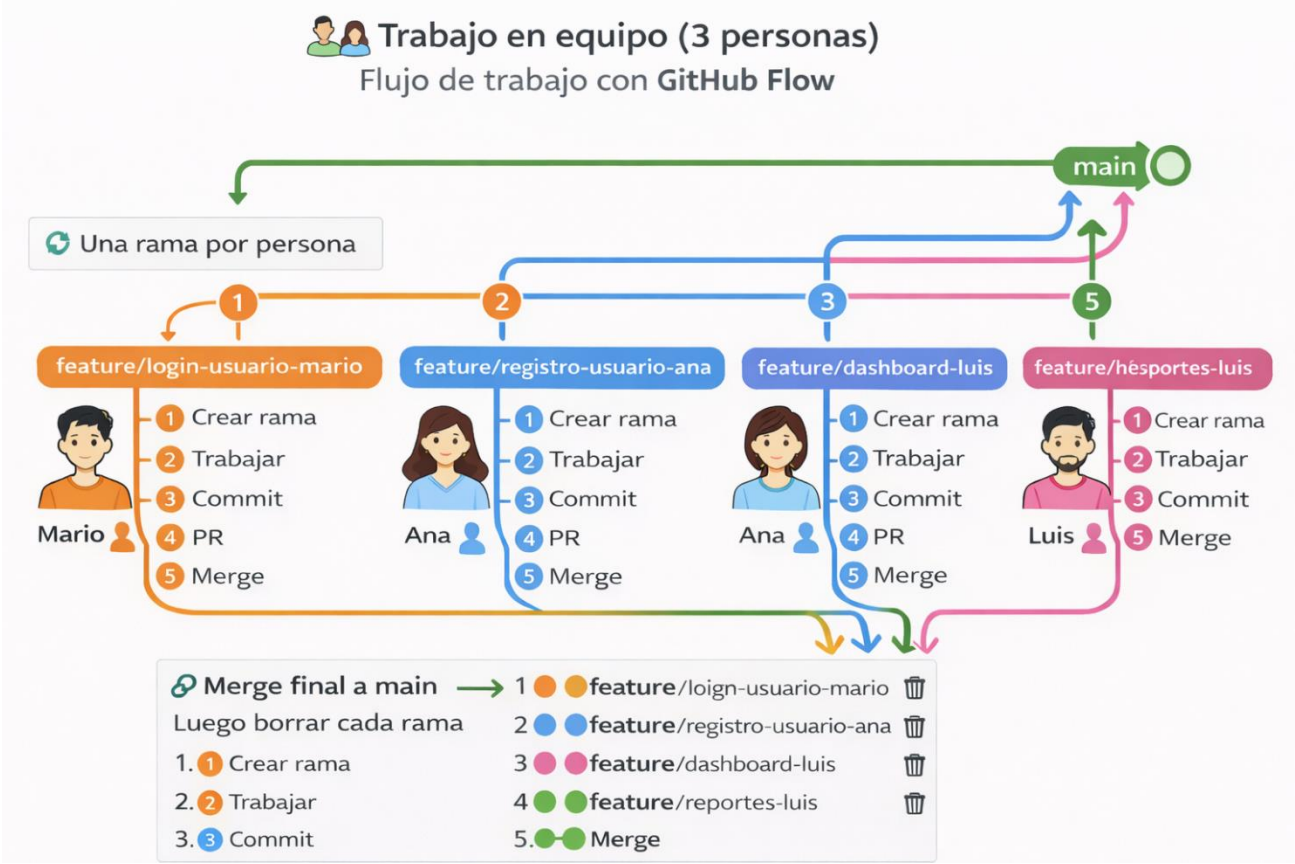
```
C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>git log --oneline -5
8f83881 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from ProfeVivitaSol/feature/mi-primera-seccion
376baf6 (origin/feature/mi-primera-seccion) docs: agrego sección de introducción al README
9aaf183 Initial commit

C:\Users\vivis\OneDrive\Desktop\practicaGithubFlow>
```

<u>Comandos de referencia</u>

Comando	¿Para qué sirve?
git status	Ver qué archivos cambiaron y cuáles están en staging
git log --oneline	Ver el historial de commits en formato compacto
git Branch	Listar ramas locales (la activa tiene *)
git switch nombre	Cambiar a una rama existente
git switch -c nombre	Crear una rama nueva y cambiar a ella
git add .	Agregar todos los cambios al staging
git commit -m 'msg'	Crear un commit con mensaje
git push origin rama	Subir una rama local a GitHub
git pull origin main	Descargar y fusionar cambios de main
git branch -d nombre	Eliminar una rama local ya integrada
git diff	Ver cambios en detalle antes de hacer commit

ORGANIZAR GITHUB EN TRABAJO DE EQUIPO



a. Cada integrante crea una rama desde main.
b. Cada integrante realiza el mismo flujo (indicados en números 1-5): crear rama, trabajar en la rama, realizar commits, aplicar PR (revisar diff), integrarlos a main por medio de Merge.
c. Cada integrante trabaja de manera independiente pero todos terminan en main. La rama principal siempre debe quedar funcional.
d. Después de merge, se eliminan las ramas, para mantener el repositorio ordenado, evitando ramas antiguas.